Proyecto parcial de Programación Orientada a Objetos

Fuga de la Prisión

Diseño e implementación de un videojuego 2D

Universidad Peruana Cayetano Heredia
Ingeniería Informática

Jose Fernando Chavez Lozano
Mickele Carlos Torres Velazco
Piero Cemei Romero Meza
Richard Aaron Soriano Cordova

Ciclo IV. Período académico 2026-II

I. Resumen ejecutivo

Videojuego 2D de escape y sigilo con cuatro niveles.

Su objetivo es demostrar POO mediante un juego funcional.

La arquitectura separa reglas, personajes e interfaz.

Aplica asociación, agregación y composición,

además de encapsulamiento, herencia y polimorfismo.

Implementa los patrones State y Observer.

II. Introducción

Contexto y objetivo

Proyecto del curso de POO.
Demostrar sus fundamentos y patrones en un juego de escape.

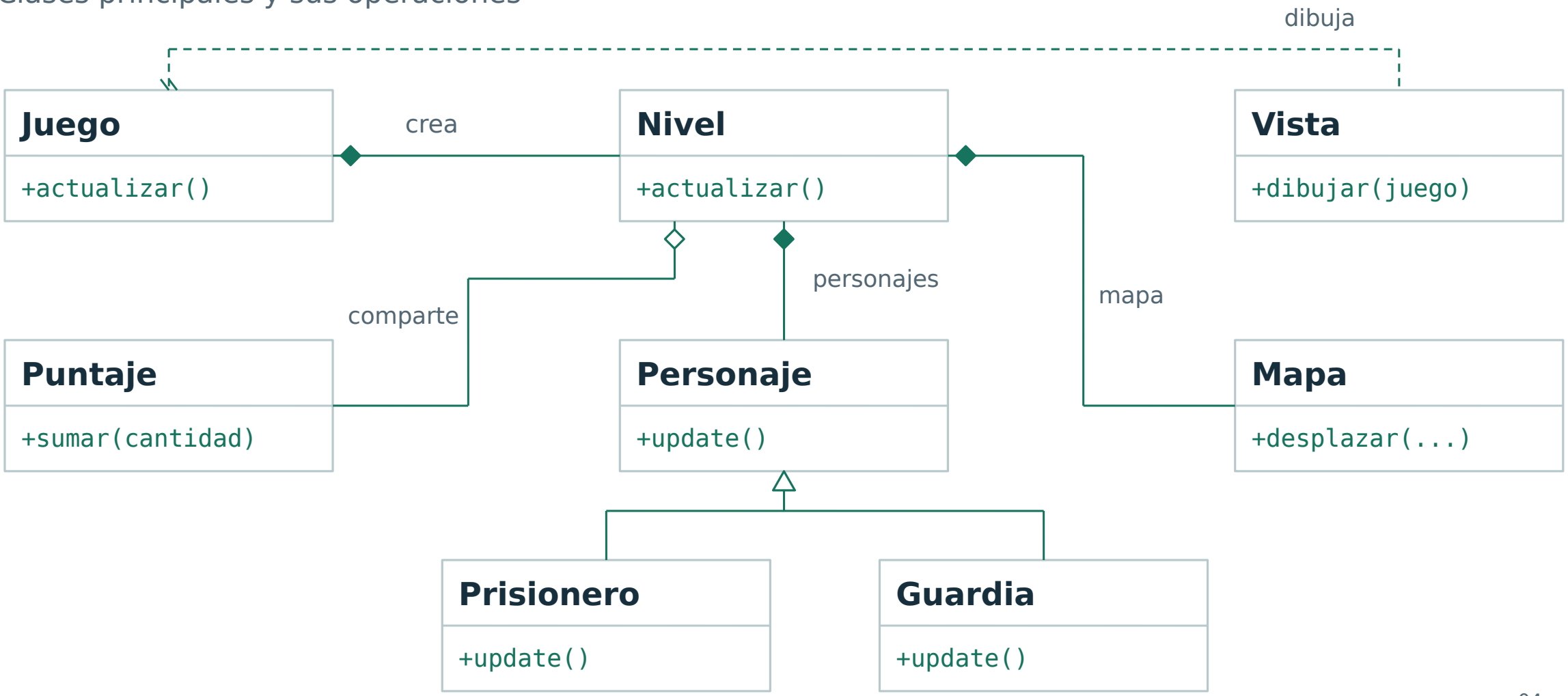
Tecnologías utilizadas

Python y Pygame: juego e interfaz.
Mermaid: diagrama UML.
Git y GitHub: control de versiones.



III. Diseño de arquitectura

Clases principales y sus operaciones



IV. Fundamentos de POO

Asociación

Personaje guarda su Nivel.

```
self.nivel = nivel
```

Agregación

Nivel recibe el Puntaje compartido de la campaña.

```
self.puntaje = puntaje
```

Composición

Nivel crea su propio Mapa.

```
self.mapa = Mapa(
    PLANOS[indice])
```

IV. Fundamentos de POO, continuación

Herencia y abstracción

Prisionero y Guardia heredan de la clase abstracta Personaje.

Polimorfismo

```
for personaje in self.personajes:
    personaje.update()
```

Encapsulamiento

Puntaje._valor se consulta con get_valor().
set_valor() valida el dato y notifica el cambio.

VI. Resultados parciales



Interfaces y funcionalidades

Apertura de celda, vigilancia, llaves y salida.
Campaña de cuatro niveles.

Uso de Git

cfaba6e Plan inicial
8917f20 Prototipo y pruebas
f3cd9fe Informe y diagramas

Ver historial del repositorio

VII. Conclusiones

Logro	Juego funcional con clases, relaciones y patrones de diseño.
Aprendizaje	Separar responsabilidades facilita probar cada regla.
Dificultad	Mantener coherentes paredes, puertas, ruido y caminos.

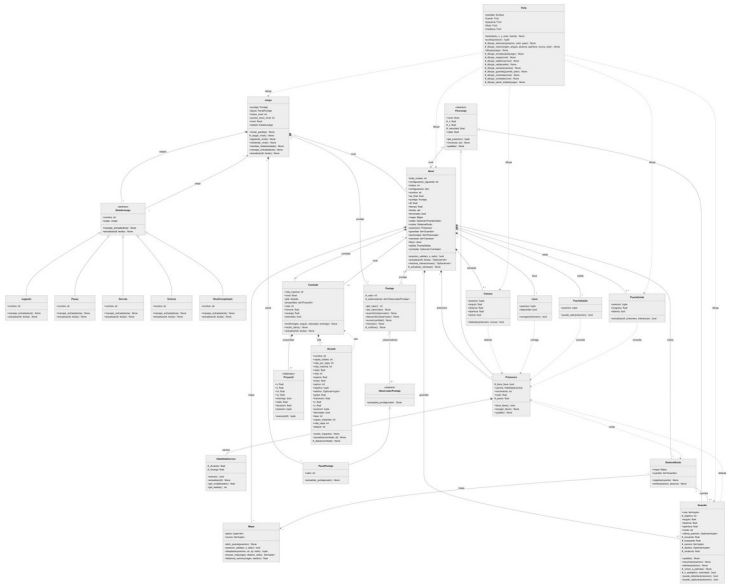
VIII. Anexos

Fragmento de código: Puntaje

```
def set_valor(self, valor):
    if type(valor) is not int or valor < 0:
        raise ValueError(
            "El puntaje debe ser un entero no negativo.")
    self._valor = valor
    self._notificar()
```

[Abrir código de Puntaje]

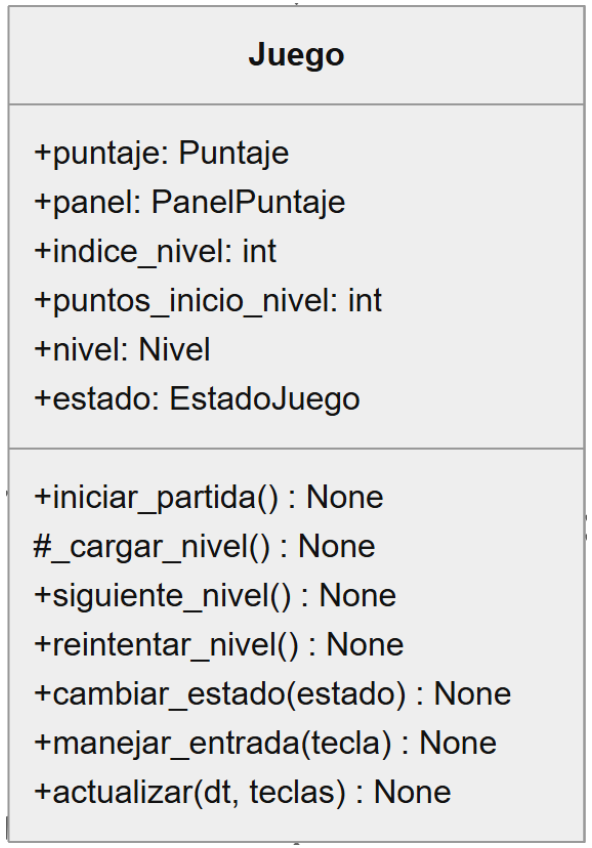
Diagrama de clases



[Abrir UML completo]

Juego

UML: recorte original



Coordina la campaña, los niveles, los puntos y el estado de la partida.

Composición: crea Puntaje, PanelPuntaje y Nivel.

Polimorfismo: delega en el EstadoJuego activo.

Código del proyecto (fragmento)

```
class Juego:

    def __init__(self):
        self.puntaje = Puntaje()
        self.panel = PanelPuntaje()
        self.puntaje.suscribir(self.panel)

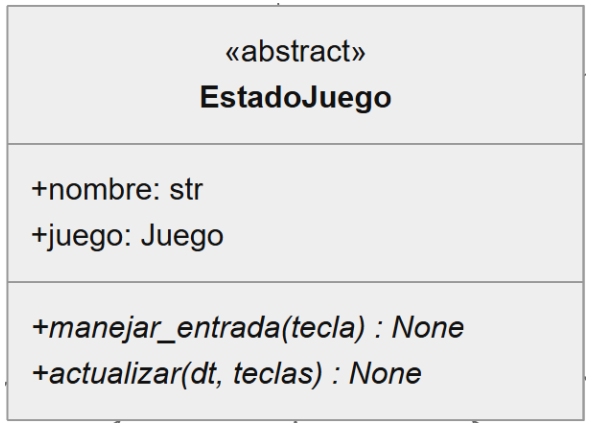
    def manejar_entrada(self, tecla):
        self.estado.manejar_entrada(tecla)
```

Abrir recorte en tamaño original

src/juego.py, clase desde línea 7

EstadoJuego

UML: recorte original



Define las operaciones comunes de los estados de la partida.

Abstracción: exige implementar entrada y actualización.

Asociación: conserva el Juego que recibe.

Código del proyecto (fragmento)

```
class EstadoJuego(ABC):

    def __init__(self, juego):
        self.juego = juego

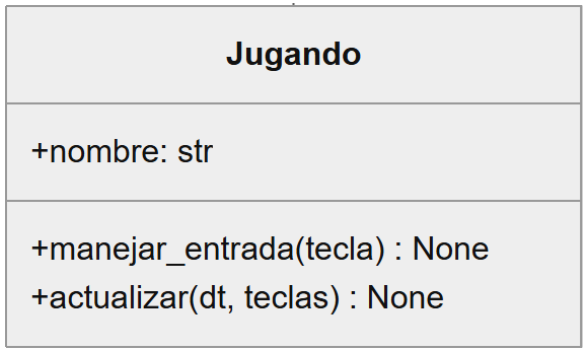
    @abstractmethod
    def actualizar(self, dt, teclas):
        pass
```

Abrir recorte en tamaño original

src/estados.py, clase desde línea 4

Jugando

UML: recorte original



Mantiene activa la simulación del nivel.

Herencia: deriva de EstadoJuego.

Polimorfismo: actualizar ejecuta la simulación.

Código del proyecto (fragmento)

```
class Jugando(EstadoJuego):

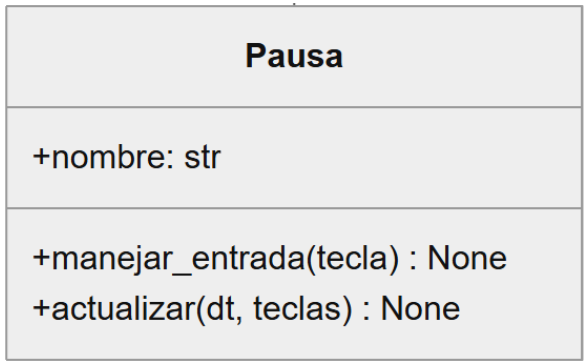
    def actualizar(self, dt, teclas):
        resultado = self.juego.nivel.actualizar(dt, teclas)
        if resultado == "derrota":
            self.juego.cambiar_estado(Derrota(self.juego))
        elif resultado == "victoria":
            self.juego.cambiar_estado(Victoria(self.juego))
        elif resultado == "nivel_completado":
            self.juego.cambiar_estado(NivelCompletado(self.juego))
```

Abrir recorte en tamaño original

src/estados.py, clase desde línea 19

Pausa

UML: recorte original



Detiene la simulación hasta que el jugador continúa.

Herencia: deriva de EstadoJuego.

Polimorfismo: actualizar deja el nivel detenido.

Código del proyecto (fragmento)

```
class Pausa(EstadoJuego):

    def manejar_entrada(self, tecla):
        if tecla == "pausa":
            self.juego.cambiar_estado(Jugando(self.juego))

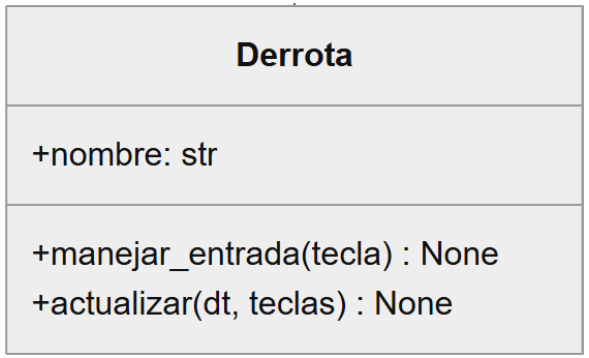
    def actualizar(self, dt, teclas):
        pass
```

Abrir recorte en tamaño original

src/estados.py, clase desde línea 38

Derrota

UML: recorte original



Abrir recorte en tamaño original

Espera la decisión del jugador después de perder.

Herencia: deriva de EstadoJuego.

Polimorfismo: la entrada permite reintentar o reiniciar.

Código del proyecto (fragmento)

```
class Derrota(EstadoJuego):

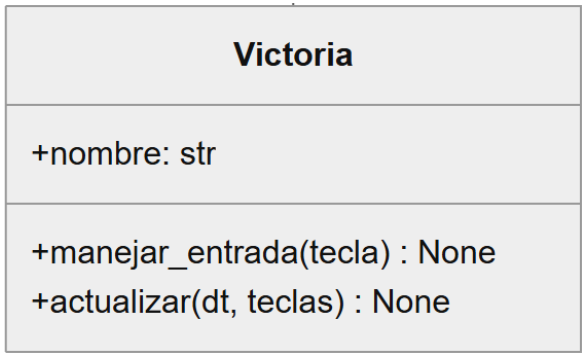
    def manejar_entrada(self, tecla):
        if tecla == "reiniciar":
            self.juego.reintentar_nivel()
        elif tecla == "nueva_partida":
            self.juego.iniciar_partida()

    def actualizar(self, dt, teclas):
        pass
```

src/estados.py, clase desde línea 49

Victoria

UML: recorte original



Representa el final de la campaña completada.

Herencia: deriva de EstadoJuego.

Polimorfismo: reiniciar inicia una campaña nueva.

Código del proyecto (fragmento)

```
class Victoria(EstadoJuego):

    def manejar_entrada(self, tecla):
        if tecla == "reiniciar":
            self.juego.iniciar_partida()

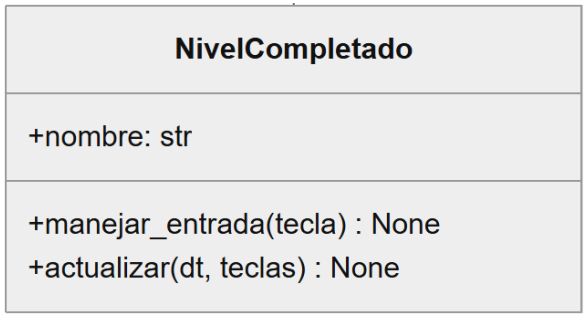
    def actualizar(self, dt, teclas):
        pass
```

Abrir recorte en tamaño original

src/estados.py, clase desde línea 62

NivelCompletado

UML: recorte original



Separa el final de un nivel del final de toda la campaña.

Herencia: deriva de EstadoJuego.

Polimorfismo: continuar carga el siguiente nivel.

Código del proyecto (fragmento)

```
class NivelCompletado(EstadoJuego):

    def manejar_entrada(self, tecla):
        if tecla == "continuar":
            self.juego.siguiente_nivel()
        elif tecla == "nueva_partida":
            self.juego.iniciar_partida()

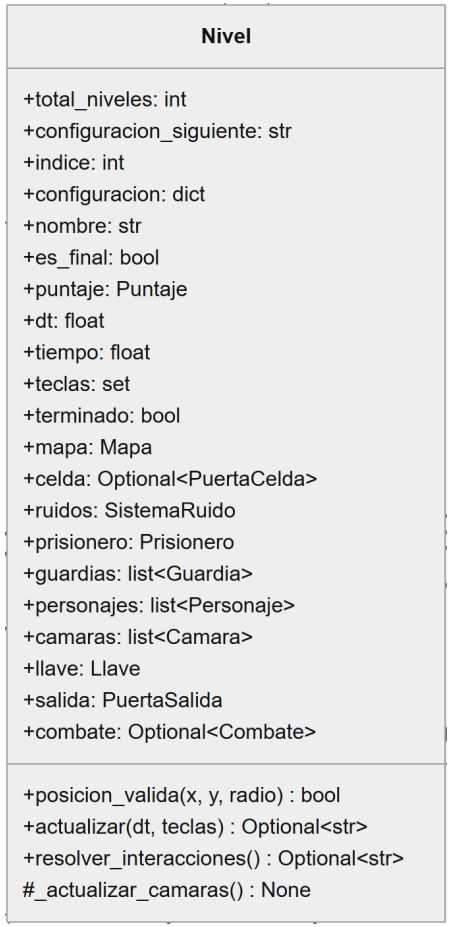
    def actualizar(self, dt, teclas):
        pass
```

Abrir recorte en tamaño original

src/estados.py, clase desde línea 73

Nivel

UML: recorte original



Abrir recorte en tamaño original

Coordina uno de los cuatro niveles y resuelve sus interacciones.

Composición: construye Mapa y personajes.

Agregación: recibe el Puntaje compartido.

Polimorfismo: llama update sin distinguir personajes.

Código del proyecto (fragmento)

```
class Nivel:

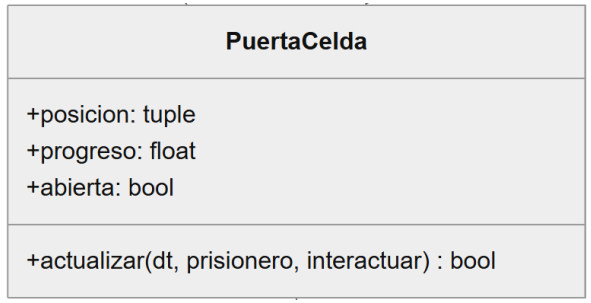
    def __init__(self, puntaje, indice=0):
        self.puntaje = puntaje
        self.mapa = Mapa(PLANOS[indice])

    def actualizar(self, dt, teclas):
        for personaje in self.personajes:
            personaje.update()
```

src/nivel.py, clase desde línea 85

PuertaCelda

UML: recorte original



Mantiene cerrada la celda inicial y controla cómo se fuerza su puerta.

Composición: Nivel crea su puerta inicial.

Dependencia: actualizar recibe al Prisionero.

Código del proyecto (fragmento)

```
class PuertaCelda:

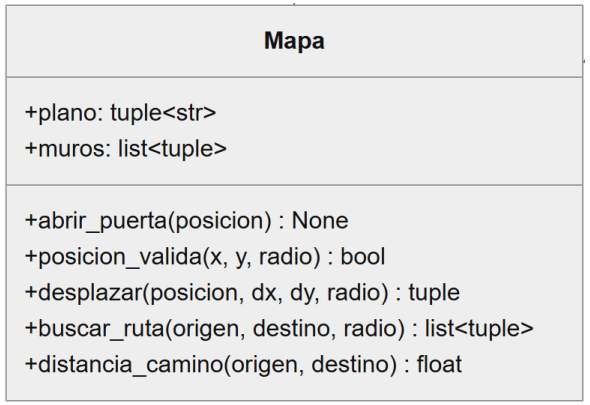
    def actualizar(self, dt, prisionero, interactuar):
        if (
            not interactuar
            or distancia(self.posicion, prisionero.get_posicion())
            > 48
        ):
            self.progreso = 0.0
            return False
```

Abrir recorte en tamaño original

src/nivel.py, clase desde línea 32

Mapa

UML: recorte original



Conserva el plano y reúne las reglas de paredes y caminos.

Composición: Nivel crea su propio Mapa.

Abstracción: ofrece colisiones y rutas mediante métodos.

Código del proyecto (fragmento)

```
class Mapa:

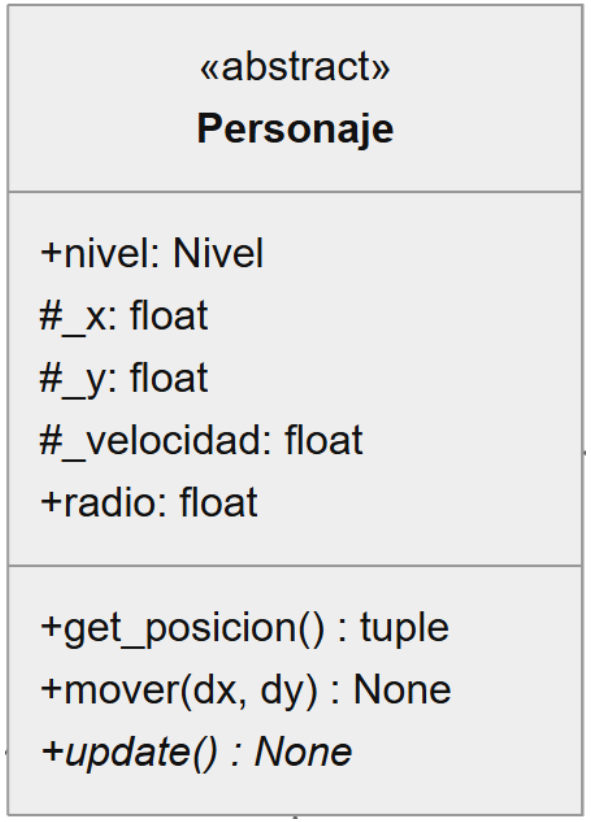
    def distancia_camino(self, origen, destino):
        ruta = self.buscar_ruta(origen, destino)
        if not ruta:
            return math.inf
        return sum(
            distancia(a, b) for a, b in zip([origen, *ruta], ruta)
        )
```

Abrir recorte en tamaño original

src/mapa.py, clase desde línea 82

Personaje

UML: recorte original



Abrir recorte en tamaño original

Reúne posición, tamaño y movimiento de los personajes.

Abstracción y herencia: base de Prisionero y Guardia.

Encapsulamiento: _x, _y y _velocidad son internos.

Asociación: guarda el Nivel que recibe.

Código del proyecto (fragmento)

```
class Personaje(ABC):

    def __init__(self, nivel, x, y, velocidad):
        self.nivel = nivel
        self._x = float(x)
        self._y = float(y)

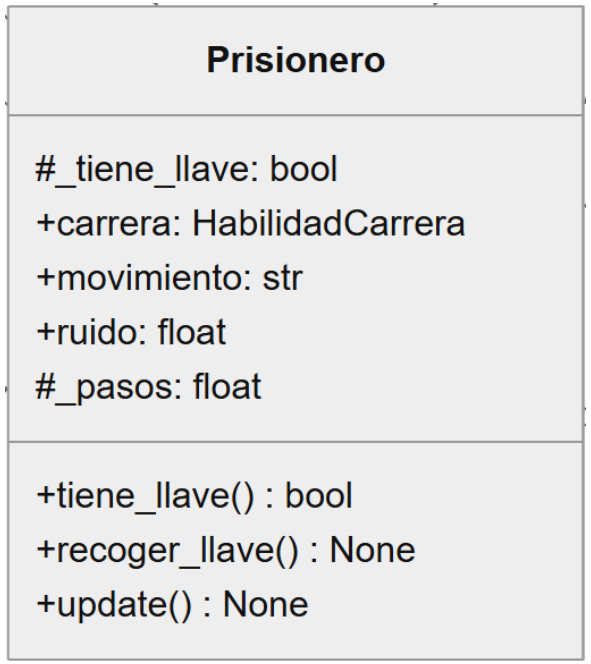
    def get_posicion(self):
        return self._x, self._y

    @abstractmethod
    def update(self):
        pass
```

src/personajes.py, clase desde línea 39

Prisionero

UML: recorte original



Es el personaje controlado por el jugador y el origen del ruido de pasos.

Herencia y polimorfismo: implementa update de Personaje.

Composición: crea su HabilidadCarrera.

Encapsulamiento: controla _tiene_llave con métodos.

Código del proyecto (fragmento)

```
class Prisionero(Personaje):

    def __init__(self, nivel, x, y):
        super().__init__(nivel, x, y, 145)
        self.carrera = HabilidadCarrera()

    def tiene_llave(self):
        return self._tiene_llave

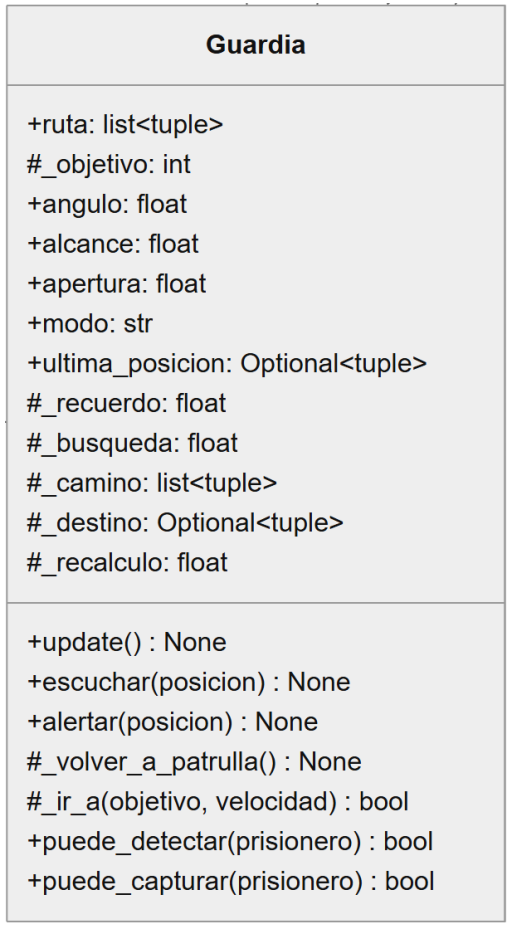
    def recoger_llave(self):
        self._tiene_llave = True
```

Abrir recorte en tamaño original

src/personajes.py, clase desde línea 58

Guardia

UML: recorte original



Abrir recorte en tamaño original

Patrulla, escucha ruidos, persigue al jugador y lo captura por contacto.

Herencia y polimorfismo: implementa update de Personaje.

Encapsulamiento: gestiona _objetivo y _camino.

Dependencia: consulta al Prisionero para detectarlo.

Código del proyecto (fragmento)

```
class Guardia(Personaje):

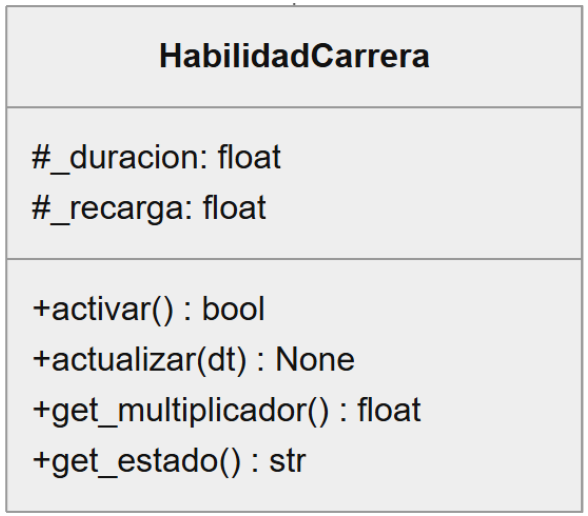
    def __init__(self, nivel, ruta):
        self._objetivo = 1
        self._camino = []

    def update(self):
        if self.puede_detectar(self.nivel.prisionero):
            self.alertar(self.nivel.prisionero.get_posicion())
        elif self.modo == "PERSIGUE" and self._recuerdo == 0:
            self.modo = "BUSCA"
```

src/personajes.py, clase desde línea 97

HabilidadCarrera

UML: recorte original



Controla el aumento temporal de velocidad del prisionero.

Encapsulamiento: gestiona _duracion y _recarga.

Composición: es creada por cada Prisionero.

Código del proyecto (fragmento)

```
class HabilidadCarrera:

    def activar(self):
        if self._duracion > 0 or self._recarga > 0:
            return False
        self._duracion = 2.0
        return True

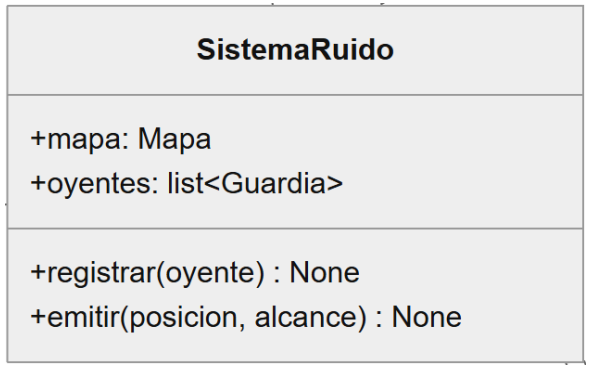
    def get_multiplicador(self):
        if self._duracion > 0:
            return 1.5
        return 1.0
```

Abrir recorte en tamaño original

src/personajes.py, clase desde línea 6

SistemaRuido

UML: recorte original



Emite los ruidos de pasos y avisa a los guardias que pueden escucharlos.

Asociación: conserva el Mapa recibido.

Agregación: registra guardias creados por Nivel.

Notificación: avisa a los oyentes que pueden escuchar.

Código del proyecto (fragmento)

```
class SistemaRuido:

    def __init__(self, mapa):
        self.mapa = mapa
        self.oyentes = []

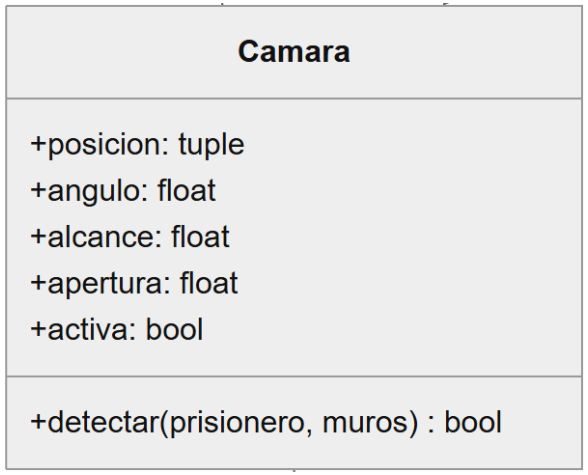
    def registrar(self, oyente):
        if oyente not in self.oyentes:
            self.oyentes.append(oyente)
```

Abrir recorte en tamaño original

src/ruido.py, clase desde línea 4

Camara

UML: recorte original



Comprueba si el prisionero está dentro de su campo de visión.

Composición: Nivel construye sus cámaras.

Dependencia: detectar recibe Prisionero y muros.

Código del proyecto (fragmento)

```
class Camara:

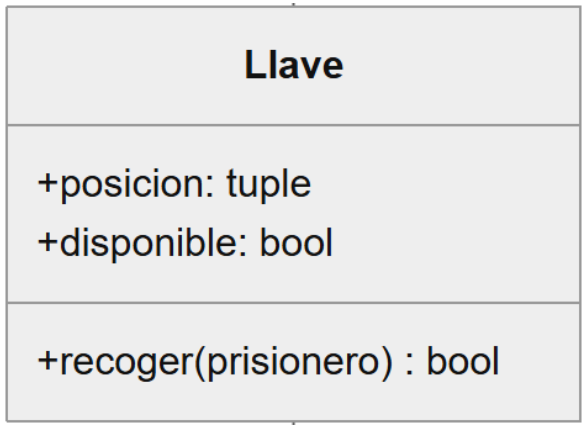
    def detectar(self, prisionero, muros):
        return dentro_de_vision(
            self.posicion,
            prisionero.get_posicion(),
            self.angulo,
            self.alcance,
            self.apertura,
            muros,
            prisionero.radio,
        )
```

Abrir recorte en tamaño original

src/nivel.py, clase desde línea 51

Llave

UML: recorte original



Abrir recorte en tamaño original

Es el objeto que habilita la salida.

Composición: cada Nivel crea su Llave.

Dependencia: recoger utiliza el Prisionero recibido.

Código del proyecto (fragmento)

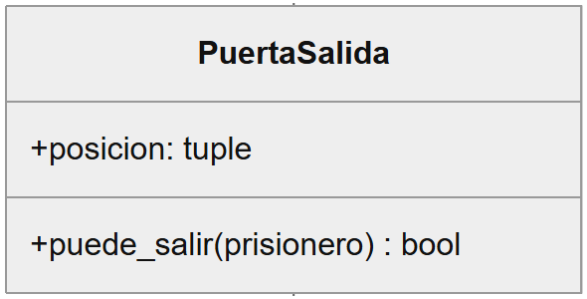
```
class Llave:

    def recoger(self, prisionero):
        if (
            self.disponible
            and distancia(
                self.posicion, prisionero.get_posicion()
            )
            < 22
        ):
            self.disponible = False
            prisionero.recoger_llave()
            return True
        return False
```

src/nivel.py, clase desde línea 64

PuertaSalida

UML: recorte original



Representa el lugar por el que se termina un nivel.

Composición: Nivel crea su PuertaSalida.

Dependencia: puede_salir consulta al Prisionero.

Código del proyecto (fragmento)

```
class PuertaSalida:

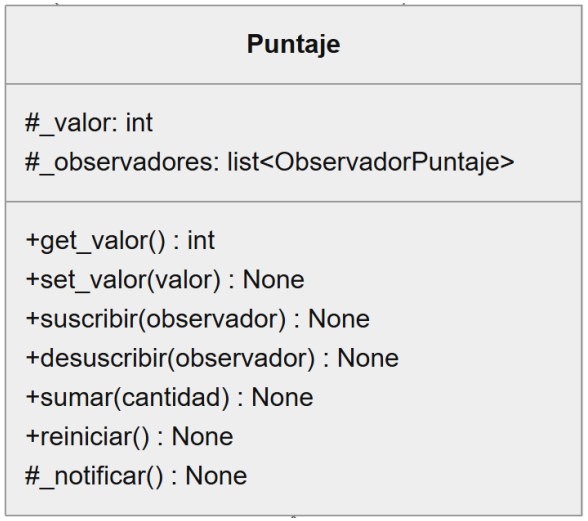
    def puede_salir(self, prisionero):
        return (
            prisionero.tiene_llave()
            and distancia(
                self.posicion, prisionero.get_posicion()
            )
            < 22
        )
```

Abrir recorte en tamaño original

src/nivel.py, clase desde línea 77

Puntaje

UML: recorte original



Abrir recorte en tamaño original

Conserva los puntos y avisa cuando cambian.

Encapsulamiento: valida los cambios de _valor.

Agregación: conserva observadores creados fuera.

Observer: notifica mediante actualizar_puntaje.

Código del proyecto (fragmento)

```
class Puntaje:

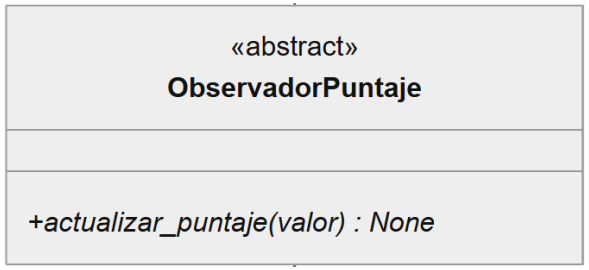
    def set_valor(self, valor):
        if type(valor) is not int or valor < 0:
            raise ValueError(
                "El puntaje debe ser un entero no negativo."
            )
        self._valor = valor
        self._notificar()

    def _notificar(self):
        for observador in tuple(self._observadores):
            observador.actualizar_puntaje(self._valor)
```

src/puntuacion.py, clase desde línea 18

ObservadorPuntaje

UML: recorte original



Define cómo debe recibir los cambios un observador del puntaje.

Abstracción: exige implementar actualizar_puntaje.

Herencia: es la base de PanelPuntaje.

Código del proyecto (fragmento)

```
class ObservadorPuntaje(ABC):

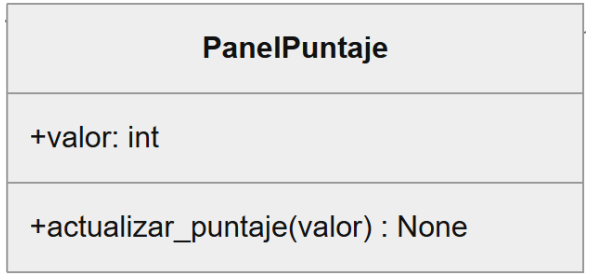
    @abstractmethod
    def actualizar_puntaje(self, valor):
        pass
```

Abrir recorte en tamaño original

src/puntuacion.py, clase desde línea 4

PanelPuntaje

UML: recorte original



Conserva el último puntaje notificado para mostrarlo en pantalla.

Herencia: deriva de ObservadorPuntaje.

Polimorfismo: implementa actualizar_puntaje.

Observer: recibe el nuevo valor que luego dibuja Vista.

Código del proyecto (fragmento)

```
class PanelPuntaje(ObservadorPuntaje):

    def __init__(self):
        self.valor = 0

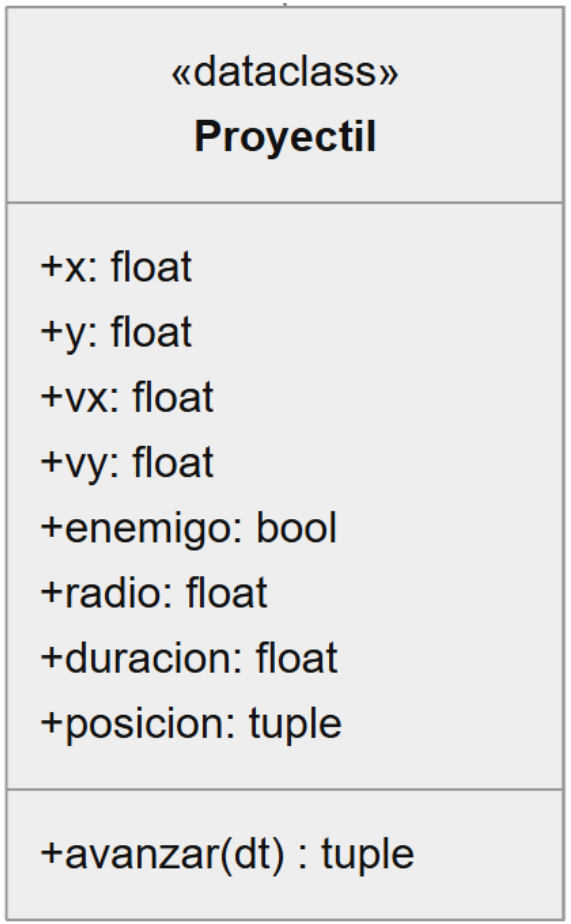
    def actualizar_puntaje(self, valor):
        self.valor = valor
```

Abrir recorte en tamaño original

src/puntuacion.py, clase desde línea 10

Proyectil

UML: recorte original



Abrir recorte en tamaño original

Guarda la posición, velocidad, tamaño y duración de un disparo.

Composición: Combate crea y conserva los disparos.

Dataclass: genera el constructor de sus datos.

Código del proyecto (fragmento)

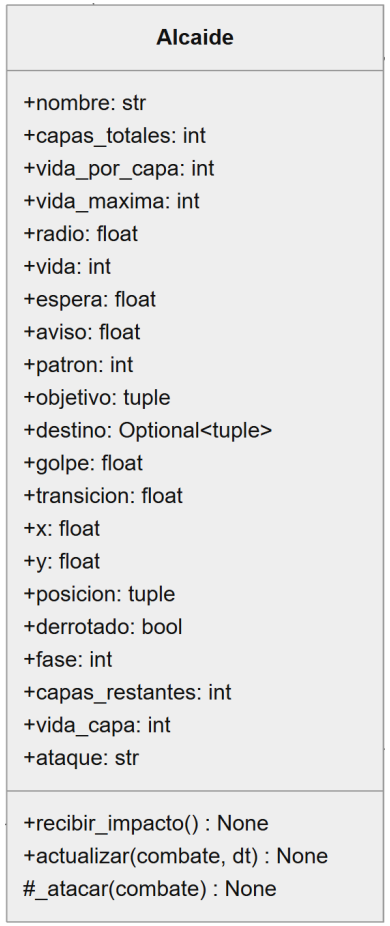
```
@dataclass
class Proyectil:
    x: float
    y: float
    vx: float
    vy: float

    def avanzar(self, dt):
        inicio = self.posicion
        self.x += self.vx * dt
        self.y += self.vy * dt
        self.duracion -= dt
        return inicio, self.posicion
```

src/combate.py, clase desde línea 8

Alcaide

UML: recorte original



Abrir recorte en tamaño original

Controla al jefe del último nivel.

Composición: Combate crea su Alcaide.

Dependencia: actualizar recibe el Combate.

Propiedades: consultan fase, posición y derrota.

Código del proyecto (fragmento)

```
class Alcaide:

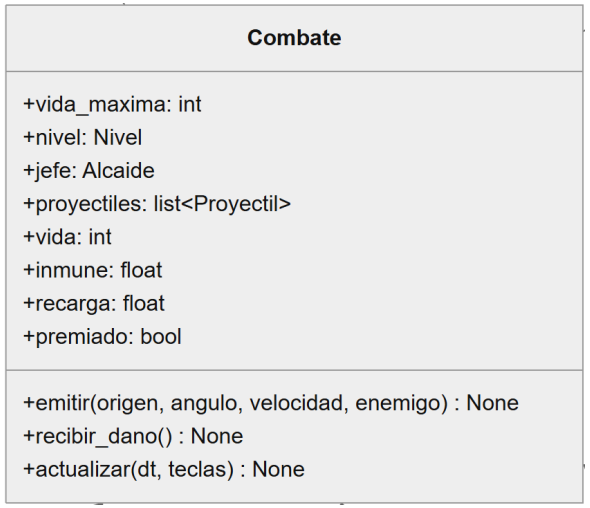
    @property
    def posicion(self):
        return self.x, self.y

    @property
    def derrotado(self):
        return self.vida <= 0
```

src/combate.py, clase desde línea 47

Combate

UML: recorte original



Resuelve los ataques, los proyectiles y la vida en la arena.

Composición: crea Alcaide y Proyectil.

Asociación: conserva el Nivel recibido.

Código del proyecto (fragmento)

```
class Combate:
    def __init__(self, nivel):
        self.nivel = nivel
        self.jefe = Alcaide()
        self.proyectiles = []

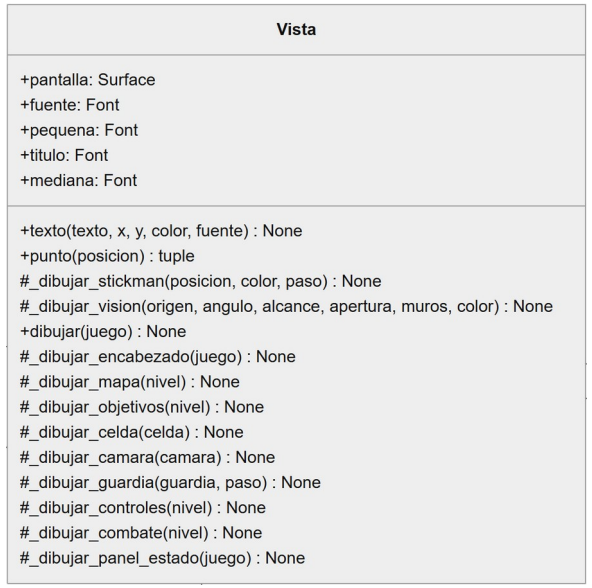
    def emitir(self, origen, angulo, velocidad, enemigo=True):
        self.proyectiles.append(Proyectil(
            *origen, math.cos(angulo) * velocidad,
            math.sin(angulo) * velocidad, enemigo,
            6 if enemigo else 5))
```

Abrir recorte en tamaño original

src/combate.py, clase desde línea 162

Vista

UML: recorte original



Abrir recorte en tamaño original

Dibuja la interfaz, los mapas y los personajes stickman con Pygame.

Asociación: conserva la pantalla que recibe.

Dependencia: dibujar consulta Juego y Nivel.

Código del proyecto (fragmento)

```
class Vista:

    def dibujar(self, juego):
        nivel = juego.nivel
        self._dibujar_encabezado(juego)
        self._dibujar_mapa(nivel)
        self._dibujar_objetivos(nivel)
```

src/vista.py, clase desde línea 12